

EELE 371 – Microprocessor HW/SW Systems

Lab 1.1 – Getting CCS Running

Objective

The objective of this lab is for you to install TI Code Composer Studio (CCS) on your own laptop and verify that it is able to program the MSP430FR2355 LaunchPad board.

Outcomes

After completing this lab, you should be able to:

- Launch CCS, create a project, enter assembly code (provided this time – in the future you will use your own), assemble, download, and run the program on the MSP430FR2355 LaunchPad board.

Deliverables

The deliverable(s) for this lab are as follows:

- You will show an instructor that you have successfully created a CCS project and run the tools from start-to-finish to get a program on the MSP430FR2355 board. You will show the instructor your CCS project and your LaunchPad board blinking LED1. The instructor will check you off.
- You will upload your main.asm file to the Lab 1.1 Assignment Folder. The instructors will verify that the program you uploaded is indeed the program that was demonstrated.

Lab Work & Demonstration

1) Install *Code Composer Studio* on your own laptop.

Note: Code Composer Studio (CCS) is a development environment where you can create assembly or C programs that are implemented on TI MCUs. The tool allows you to assemble or compile your programs, download them to a TI MCU (ie: MSP430 located on the Launchpad), and then run the program. Additionally, you can observe the details of the code's operation to verify that your program is operating as expected. CCS does not require a license and can run on Windows, Mac, or Linux (although some Mac users experience more difficulties than Windows/Linux users).

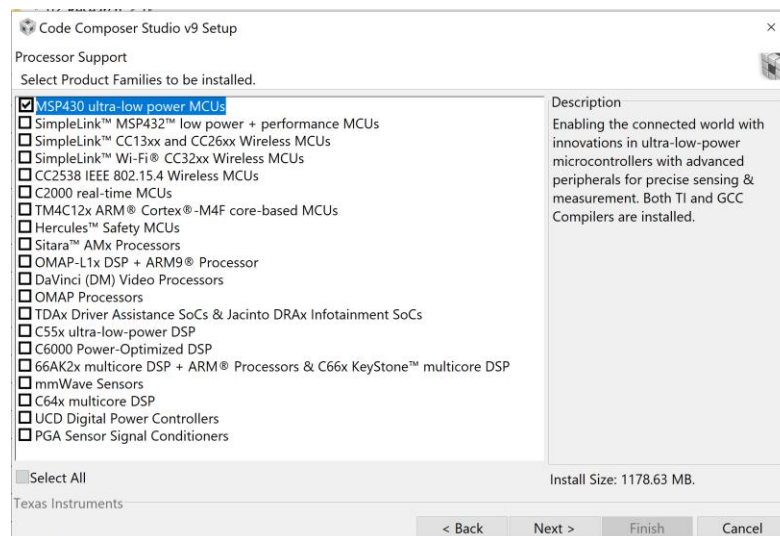
BEFORE plugging the Launchpad into your computer:

- Go to the following download website: http://software-dl.ti.com/ccs/esd/documents/ccs_downloads.html.
- Scroll down to the Version X Downloads section (see example figure on the next page). Download **version 12.8.1**. Note: Newer versions such as **version 20.0.0** are NOT compatible with this course! Click on your operating system. You will begin downloading a ~869M zip file...it might take a while.
- Here is an example of what to expect. At the time of writing this document, the latest version was #9 (see title) and you can select your operating system in the 4th column.

Code Composer Studio Version 9 Downloads				
Release	Build #	Date	Download	Notes
9.1.0	9.1.0.00010	Jun 24, 2019	Off-line Installers: Windows 64-bit-only MD5 Linux MD5 - 64-bit only MacOS MD5 Manifest	• New/Notable In This Release (9.1.0.00004): • Release notes • Windows support: Code Composer Studio is now supported only on 64bit Windows machines. • Bugfixes for IDE, DVT, Debugger • CCS User's Guide • MCU Compilers LTS 18.12.2 • C6000 Compiler 8.3.3 • MSP GCC v8.2.0.52 • SysConfig Core v1.1.792 • XDCTools 3.55.02.22 GA • Device support updates ◦ C2000 v5.0.0 ◦ mmWave v1.1.2 ◦ Sitara v1.4.7 ◦ Shared support v1.1.3 ◦ MSP430 ◦ MSP432 • TI EMU19_M05 v8.2.0.00004

Please note: different app and OS versions will require slightly different steps. What follows are general guidelines.

- Once downloaded, unzip the file, if necessary, and browse to the install file (on windows this is the "ccs_setup_win64_9.1.0.00010.exe"). Double click on the install file to begin installation unless it starts automatically.
- You will see a series of windows asking if you accept the terms of agreement, etc. Click yes or next on all of them.
- When you get to the point where it wants you to select a location for the install, allow it to install to the default location on your computer (for Windows this is: c:\ti\ccs910).
- You will be asked which type of installation you'd prefer Custom or Full, choose **Custom**. Then, you will be asked which CCS family-support to install. Choose the "**MSP430 ultra-low power MCUs**".



- Accept the rest of the default settings and finish the installation. The installation should complete. Depending on your operating system, you may need to restart. Do as instructed. **This isn't a fast install, so don't worry if it is taking 10+ minutes.**

2) Once you have successfully installed CCS, plug in the MSP430FR2355 LaunchPad Board

The first time you plug in the LaunchPad board into your laptop it will install the USB drivers. *Don't plug in your board until CCS is installed.*

- Plug the board in using the provided USB cable and allow the driver installation to complete. (sometimes using a longer cable can cause communication difficulties)

3) Launch Code Composer Studio

- Start CCS. On Windows this is done using the Start menu: *Start → Texas Instruments → Code Composer Studio 9.x.x (or using the shortcut icon on the desktop).*

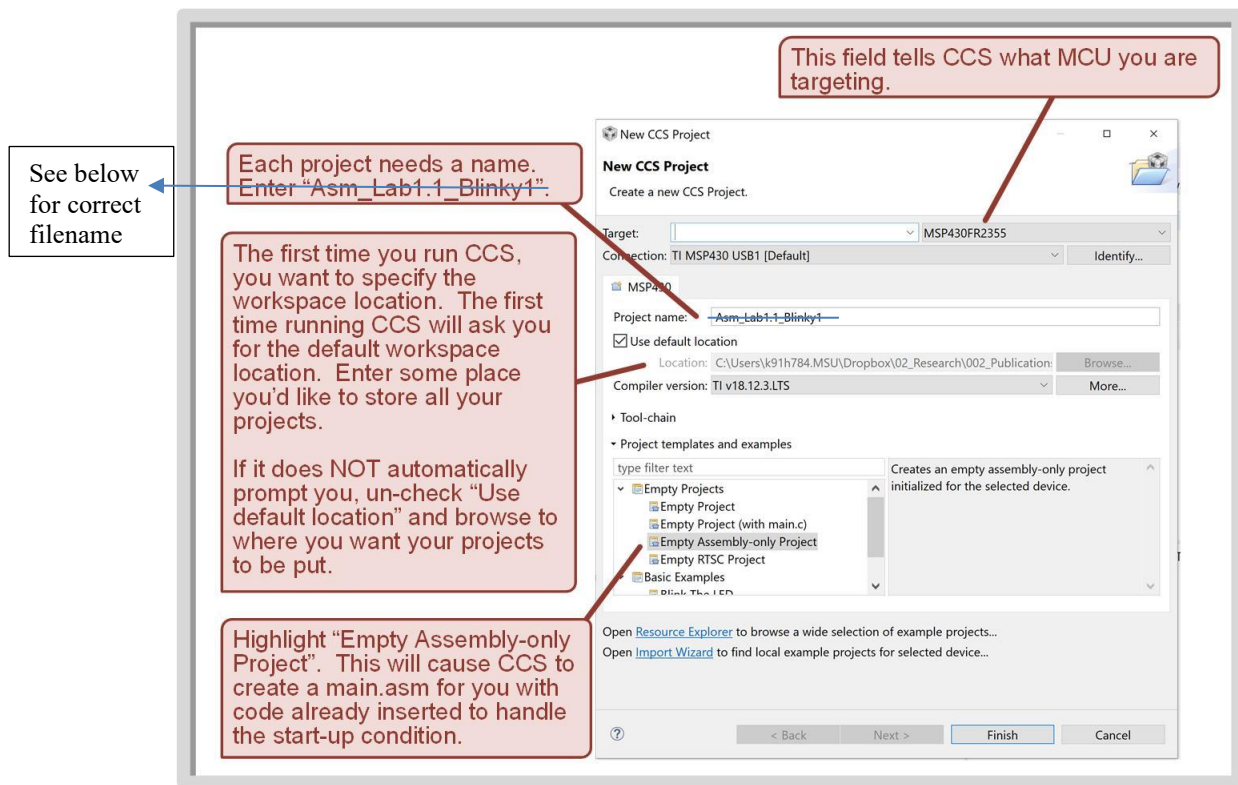
4) Define the workspace (only necessary the very first time you launch CCS)

Note: CCS uses the concept of a “Workspace” to hold all of your projects. The workspace is a folder on your computer where you can have many projects. In this class, you will define one workspace location. Then, for each example, lab, and practical you will create a new project. By the end of the semester, you will have a workspace with potentially dozens of projects. Keeping these projects in the same workspace makes it easy to refer back to prior labs to recall how you implemented something. The workspace location is defined the first time you create a project in CCS.

- If prompted, browse to Documents/EELE371/Workspace (you need to create these folders).
- If not prompted, File → Switch Workspace → Browse to the same folder

5) Create a new CCS Project

- Navigate to File → New → CCS Project (or use the icon at the top of the Getting Started page). The ‘New CSS Project’ screen will appear. You will need to fill in some fields of the screen.

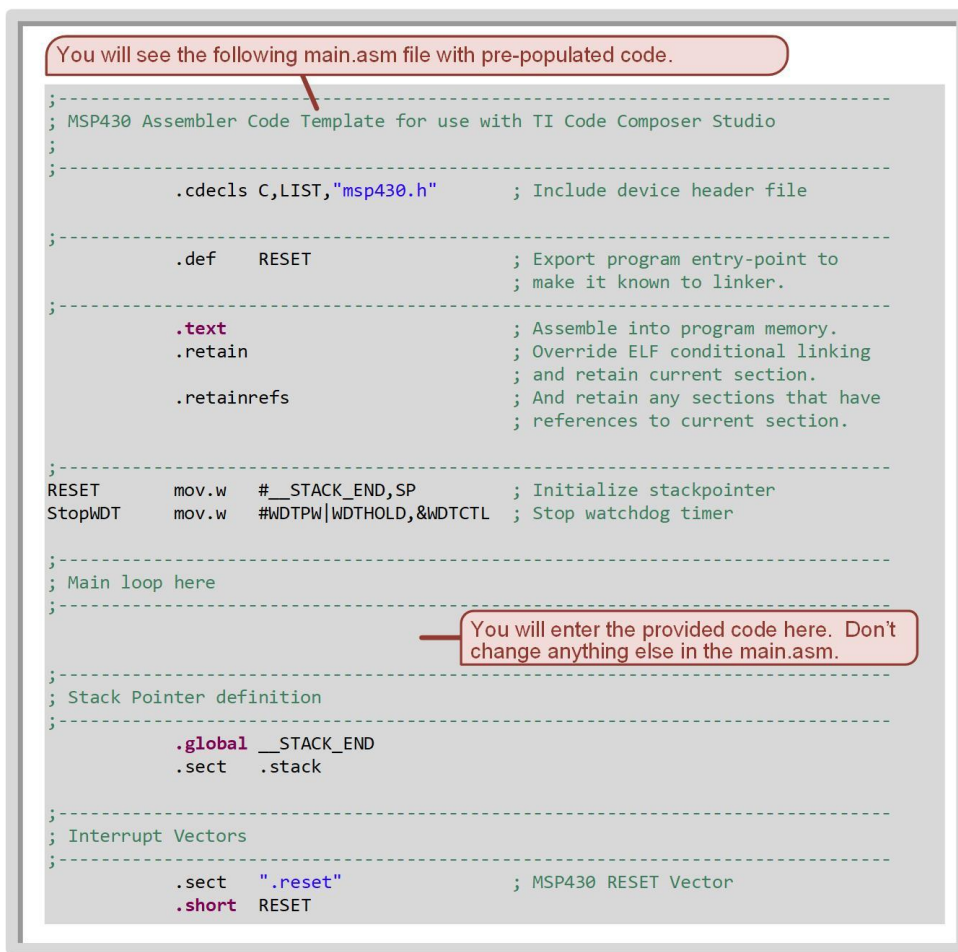


- In the “Target Field”, select: **MSP430FRxxx Family** and next to that **MSP430FR2355**
- In the “Project name” field, enter: **Lab1.1_ASM_Blinky1_LastName**
- Browse to the class workspace folder you created above (unless it correctly was assigned as the default)
- In “Project templates and examples”, select: **Empty Assembly-only Project**.
- Click “Finish”.

5. Inspect the main.asm file that was created for you.

- CCS will automatically create this main.asm file for you and pre-populate it with code necessary for the start-up condition. There is a header (e.g., a type of ‘comment’) on line 24 (approx.) where you will eventually insert your own main program.

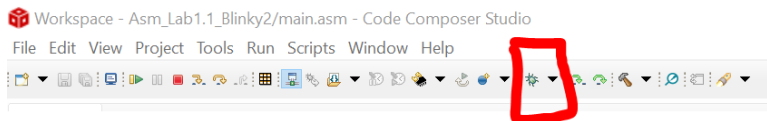
- You should see a main.asm that looks like this:



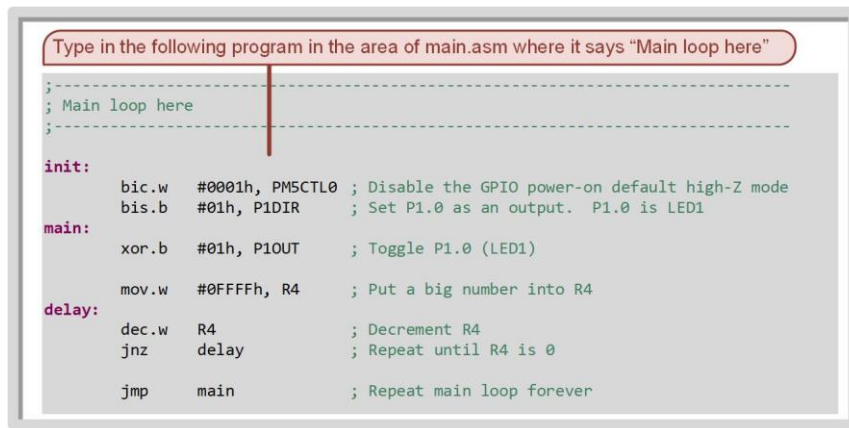
- You should navigate to the project folder on the left side of the CCS window and rename this file to *main_Lab1.1.asm*. In the near future you will have many main.asm files open at once, if they are all named the same, it is difficult to find what you will be looking for!

6. Force installation of proper libraries.

- This step should only have to be completed the first time you set up the operating environment.
- Click on the debug icon and verify that the code compiles and downloads to the hardware.



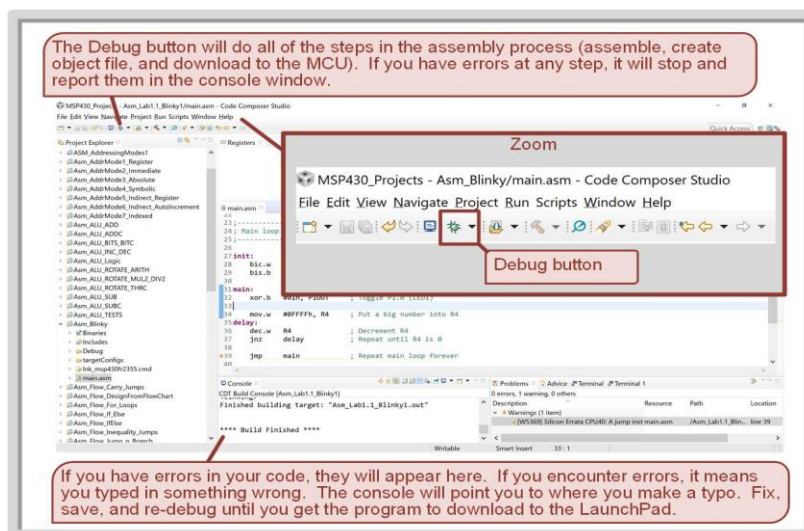
7. Enter (type) the provided code to make LED1 blink.



- Pay close attention to the alignment, font, capitalization, and punctuation of this code!
-

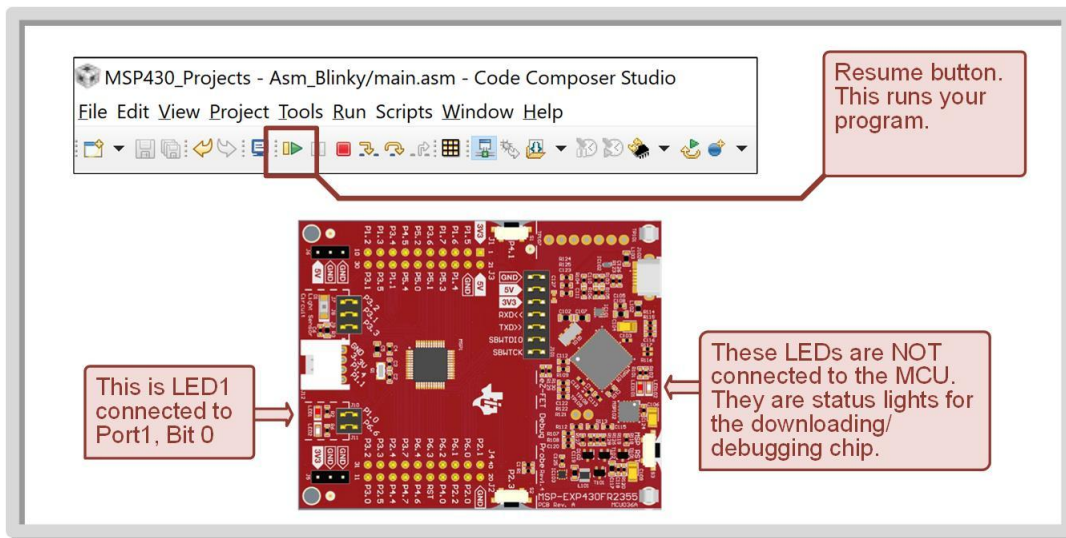
8. Save and Debug Your Program.

- Save your main.asm using CNTL-S or using the menus File → Save.
- Click on the “Debug” button to assemble your program, create the file for the MCU, and download it to the LaunchPad board. Fix errors as necessary.



9. Run Your Program.

- After downloading your program, it will be located on the LaunchPad MCU, but it won't be running. We have to tell CCS to allow the code to run. Click on the "Resume" button in CCS. You should see LED1 blinking.



10. Modify your code

This class will require you to think critically (possibly more than any other course you have taken so far). It is understood that you probably do not know how to do everything you will be asked to do before having to do it – this is OK! The majority of engineering is ‘figuring it out.’ If you don't know how to do something, TRY anything – you are *encouraged* to play around with your new device!

You will be asked many times this semester to modify (or eventually build) code that you have not been instructed on. The easy way to do this is to ask your neighbor – but please try to avoid this! While it won't be considered ‘cheating’ necessarily, you are missing out on the learning opportunity if you don't do the work for yourself.

Some hints on how to get started:

- Review the existing code, make sure you understand each step
 - Run the debugger step by step and observe how the code works
 - Write small pieces of test code to test your ideas, examples:
 - If you are not exactly sure what a value is doing, rather than try to figure out the exact right new value, change the old to something random and observe what happens
 - If you are trying to make if/then decisions, write the decision code, but don't try to code the appropriate results – first check to see if you are make the decision correctly by using flashing LEDs (ie: if you want to do something only when a test value is greater than 2, write the code to test >2, if it is >2 flash an LED slowly, if it is not, flash the LED quickly; once you know you move through the code correctly, you can write more complicated resulting code).
 - DON'T delete working code!!!! You might want to get back to something operational.
 - Comment out a line or two of working code and retype small changes
 - For larger changes, create a new version of code (save the old, work on the new)
- Required modification: change the speed at which the LED flashes
- Challenge modification: modify the flashing profile so that the LED stays lit for a long pulse, but is off only for a short pulse.

11. Demo your lab to this point with an instructor

- Call over an instructor and show them your LED blinking at a faster rate (required modification).
- It is not necessary to demo the challenge modification if you ran out of time today.

12. Upload your main.asm file to the Lab 1.1 Assignment Folder

- Log into the Canvas site for the course. Go the Module/Module 1 – Getting Started/Lab 1.1 Results. Upload your main.asm file.

Note: This is the point where you need to know where your workspace is located. Remember that you set the location of the workstation when you created the project. Navigate there, find the .asm file, and drag into the Canvas Assignment box.